

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1. A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

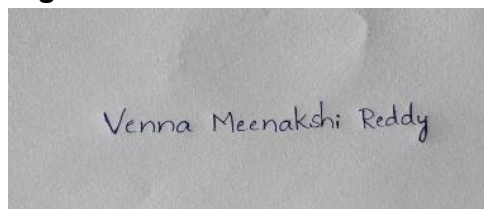
3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I VENNA MEENAKSHI REDDY(192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:A photograph of a piece of paper with the handwritten signature "Venna Meenakshi Reddy" in blue ink.**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1.A hospital management system requires a SOAP-based web service to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

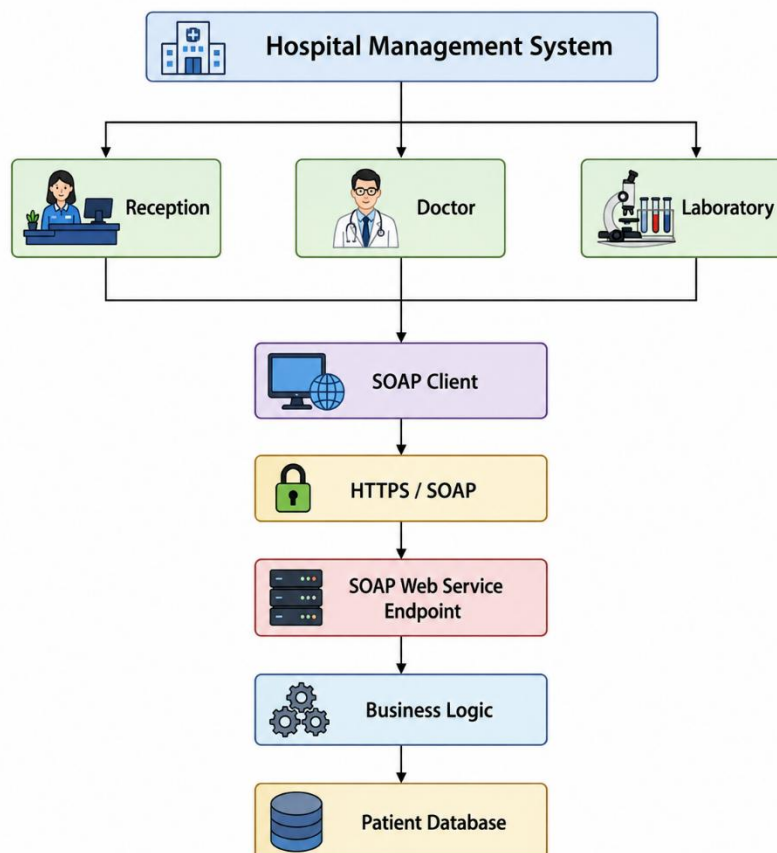
Design a SOAP-based web service for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

1.1 Introduction

A hospital management system must allow different departments such as reception, outpatient department, laboratory, pharmacy, billing, and medical records to exchange patient information securely and consistently.

A SOAP (Simple Object Access Protocol) web service is suitable because it provides a standardized XML-based communication mechanism and supports security, reliability, formal contracts, and interoperability between systems developed using different programming languages and platforms.

Proposed architecture



Each hospital department acts as a SOAP client, while the hospital server exposes SOAP operations through a service endpoint.

Example endpoint:

<https://hospital.example.com/services/PatientService>

1.2 Main SOAP Operations

The hospital service can provide the following operations:

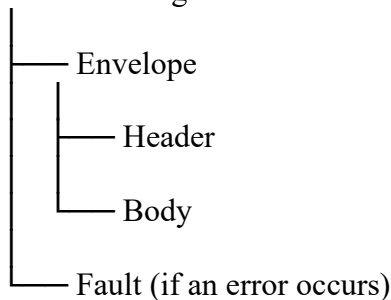
Operation	Purpose	Input	Output
addPatient	Registers a new patient	Patient details	Patient ID
getPatient	Retrieves patient details	Patient ID	Patient information
updatePatient	Updates patient information	Patient details	Status
scheduleAppointment	Creates an appointment	Patient ID, doctor, date	Appointment ID
getAppointments	Retrieves appointments	Patient ID	Appointment list
addMedicalRecord	Adds medical information	Patient ID, record	Status
getMedicalRecords	Retrieves medical records	Patient ID	Medical records

These operations provide separation between the service interface and the internal database implementation.

1.3 Structure of a SOAP Message

A SOAP message consists mainly of:

SOAP Message



1. Envelope

The Envelope is the root element of every SOAP message. It identifies the document as a SOAP message and defines the SOAP namespace.

2. Header

The Header contains optional information such as:

- Authentication
- Authorization
- Security tokens
- Transaction information
- Message identifiers
- Routing information

For a hospital system, security information can be included in the header.

3. Body

The Body contains the actual application request or response.
For example:

```
<soap:Body>
  <addPatient>
    <patient>
      <name>Ravi Kumar</name>
      <age>45</age>
      <gender>Male</gender>
      <phone>9876543210</phone>
    </patient>
  </addPatient>
</soap:Body>
```

1.4 Example SOAP Request

A client registering a patient can send:

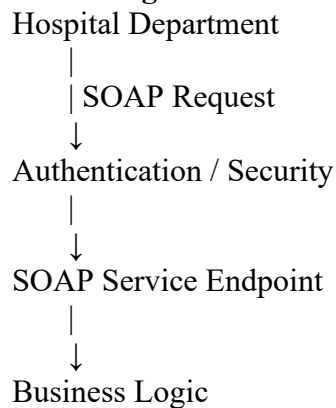
```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hs="http://hospital.example.com/patient">

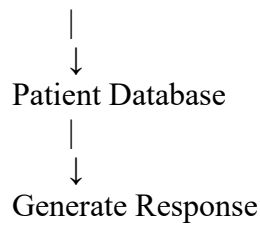
  <soap:Header>
    <hs:Authentication>
      <hs:Token>ABC123XYZ</hs:Token>
    </hs:Authentication>
  </soap:Header>

  <soap:Body>
    <hs:addPatient>
      <hs:patient>
        <hs:name>Ravi Kumar</hs:name>
        <hs:age>45</hs:age>
        <hs:gender>Male</hs:gender>
        <hs:phone>9876543210</hs:phone>
      </hs:patient>
    </hs:addPatient>
  </soap:Body>

</soap:Envelope>
```

Processing





1.5 SOAP Response

If the patient is successfully registered:

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hs="http://hospital.example.com/patient">

  <soap:Body>
    <hs:addPatientResponse>
      <hs:status>SUCCESS</hs:status>
      <hs:patientId>P10045</hs:patientId>
      <hs:message>Patient registered successfully</hs:message>
    </hs:addPatientResponse>
  </soap:Body>

</soap:Envelope>
```

The client extracts the patientId and uses it for future operations.

1.6 Retrieving Medical Records

A doctor can request medical records using:

```
<soap:Body>
  <hs:getMedicalRecords>
    <hs:patientId>P10045</hs:patientId>
  </hs:getMedicalRecords>
</soap:Body>
```

The server validates the request, checks authorization, retrieves the records, and returns:

```
<soap:Body>
  <hs:getMedicalRecordsResponse>

    <hs:patientId>P10045</hs:patientId>

    <hs:record>
      <hs:date>2026-08-20</hs:date>
      <hs:doctor>Dr. Kumar</hs:doctor>
      <hs:diagnosis>Fever</hs:diagnosis>
      <hs:treatment>Medication prescribed</hs:treatment>
    </hs:record>

  </hs:getMedicalRecordsResponse>
</soap:Body>
```

1.7 SOAP Fault Handling

If the patient ID does not exist, the service should not return an ordinary success response. It should generate a SOAP Fault.

```
<soap:Fault>  
  <faultcode>soap:Client</faultcode>  
  <faultstring>Patient ID does not exist</faultstring>  
</soap:Fault>
```

This allows the client to distinguish between successful and unsuccessful operations.

1.8 Security and Reliability

SOAP is particularly appropriate for healthcare because patient information is sensitive.

Security

The system can use:

- HTTPS/TLS for encrypted communication.
- WS-Security for message-level security.
- Authentication tokens or certificates.
- Digital signatures to verify message integrity.
- Encryption of sensitive SOAP message content.
- Role-based authorization for doctors, nurses, receptionists, etc.

For example:

Receptionist → Registration information

Doctor → Medical records

Laboratory → Test results

Pharmacy → Prescription information

Access should be restricted according to the user's role.

Reliability

SOAP supports enterprise-level mechanisms such as:

- Standardized fault handling.
- Reliable messaging mechanisms.
- Formal service contracts through WSDL.
- Transaction-oriented processing.
- XML validation.
- Interoperability between Java, .NET, Python, and other platforms.

Conclusion

SOAP is appropriate for the hospital system because it provides a structured, contract-based, secure, and interoperable communication mechanism. The combination of SOAP, HTTPS, WS-Security, WSDL, authentication, authorization, and fault handling makes the system suitable for exchanging critical patient information.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use WSDL (Web Services Description Language) to describe the service.

Design the WSDL structure for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

2.1 Introduction

WSDL (Web Services Description Language) is an XML-based language used to formally describe a web service.

It tells external applications:

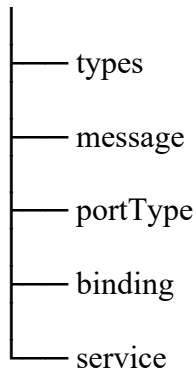
- What operations are available.
- What data must be sent.
- What data will be returned.
- Which protocol is used.
- Where the service is located.

Therefore, WSDL acts as a contract between service provider and service consumer.

2.2 Basic WSDL Structure

A WSDL document can be represented as:

WSDL



The relationship is:

Types



Messages



PortType



Binding



Service

2.3 1. Types

The types section defines the data structures exchanged by the service.
For example:

<types>

```
<xsd:schema
  targetNamespace="http://example.com/data">

  <xsd:element name="getDataRequest">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="id"
          type="xsd:int"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>

</xsd:schema>
```

</types>

It specifies that getDataRequest contains an integer id.

2.4 2. Messages

The message section defines the actual input and output messages.
Example:

```
<message name="GetDataRequest">
  <part name="parameters"
    element="tns:getDataRequest"/>
</message>
```

```
<message name="GetDataResponse">
  <part name="result"
    element="tns:getDataResponse"/>
</message>
```

Here:

GetDataRequest → Input

GetDataResponse → Output

2.5 3. PortType

portType defines the abstract interface of the service.
For example:

```
<portType name="DataServicePortType">

  <operation name="getData">
    <input message="tns:GetDataRequest"/>
    <output message="tns:GetDataResponse"/>
  </operation>

  <operation name="updateData">
    <input message="tns:UpdateDataRequest"/>
  </operation>
```

```
    <output message="tns:UpdateDataResponse"/>
  </operation>
```

```
</portType>
```

This tells the client that the service provides two operations:

```
getData()
```

```
updateData()
```

It does not yet specify how these operations are transported.

2.6 4. Binding

The binding section specifies how the abstract interface is implemented.

For example, SOAP over HTTP:

```
<binding
  name="DataServiceSoapBinding"
  type="tns:DataServicePortType">

  <soap:binding
    style="document"
    transport="http://schemas.xmlsoap.org/soap/http"/>

  <operation name="getData">
    <soap:operation
      soapAction="getData"/>

    <input>
      <soap:body use="literal"/>
    </input>

    <output>
      <soap:body use="literal"/>
    </output>
  </operation>
```

```
</binding>
```

The binding specifies:

- SOAP is used.
- HTTP is the transport.
- Document/literal messaging is used.
- SOAP action identifies the operation.

2.7 5. Service

The service element provides the actual endpoint location.

```
<service name="DataService">

  <port
    name="DataServicePort"
    binding="tns:DataServiceSoapBinding">

    <soap:address
      location="https://example.com/services/DataService"/>
```

</port>

</service>

The client therefore knows where to send SOAP requests.

2.8 Complete Simplified WSDL

A simplified WSDL structure is:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<definitions
```

```
  name="DataService"
```

```
  targetNamespace="http://example.com/data"
```

```
  xmlns:tns="http://example.com/data"
```

```
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
```

```
  xmlns="http://schemas.xmlsoap.org/wsdl/">
```

```
<!--1. Data Types -->
```

```
<types>
```

```
  <xsd:schema
```

```
    targetNamespace="http://example.com/data">
```

```
    <xsd:element name="getDataRequest">
```

```
      <xsd:complexType>
```

```
        <xsd:sequence>
```

```
          <xsd:element name="id"
```

```
            type="xsd:int"/>
```

```
        </xsd:sequence>
```

```
      </xsd:complexType>
```

```
    </xsd:element>
```

```
  </xsd:schema>
```

```
</types>
```

```
<!--2. Messages -->
```

```
<message name="GetDataRequest">
```

```
  <part name="parameters"
```

```
    element="tns:getDataRequest"/>
```

```
</message>
```

```
<message name="GetDataResponse">
```

```
  <part name="result"
```

```
    element="tns:getDataResponse"/>
```

```
</message>
```

```
<!--3. Interface -->
```

```
<portType name="DataServicePortType">
```

```
  <operation name="getData">
```

```

        <input message="tns:GetDataRequest"/>
        <output message="tns:GetDataResponse"/>
    </operation>

</portType>

<!--4. Communication Protocol →
<binding
    name="DataServiceBinding"
    type="tns:DataServicePortType">

    <soap:binding
        style="document"
        transport="http://schemas.xmlsoap.org/soap/http"/>

</binding>

<!--5. Service Endpoint →
<service name="DataService">

    <port
        name="DataServicePort"
        binding="tns:DataServiceBinding">

        <soap:address
            location="https://example.com/services/DataService"/>

    </port>

</service>

</definitions>

```

2.9 WSDL Component Relationship

WSDL Component	Function
types	Defines data structures
message	Defines input/output messages
portType	Defines abstract operations
binding	Defines communication protocol and message format
service	Defines actual service endpoint

A useful way to remember this is:

TYPES

"What data?"



MESSAGE

"What messages?"



PORTTYPE

"What operations?"



BINDING

"How do we communicate?"



SERVICE

"Where is the service?"

2.10 Importance of WSDL in Service Integration

WSDL is important because it provides a machine-readable contract.

An external system can obtain the WSDL and automatically understand:

Available Operations



Required Inputs



Expected Outputs



Communication Protocol



Endpoint Address

This reduces ambiguity between service providers and consumers.

Benefits

1. **Interoperability** – different programming languages can communicate.
2. **Loose coupling** – clients depend on the service contract rather than internal implementation.
3. **Automation** – development tools can generate client/server code from WSDL.
4. **Standardization** – communication follows a defined structure.
5. **Maintainability** – service changes can be managed through versioned contracts.
6. **Integration** – external enterprise systems can discover how to consume the service.

Conclusion

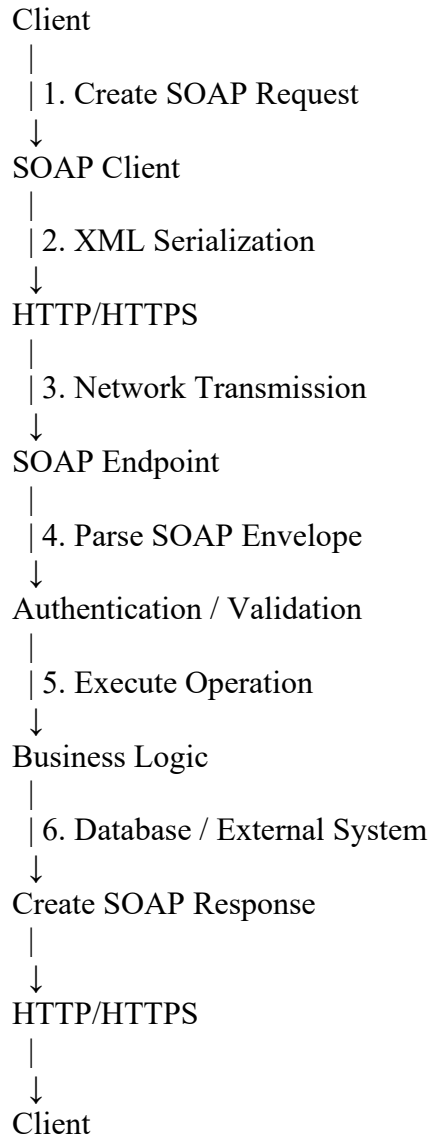
WSDL provides the formal interface contract required for reliable enterprise service integration. It separates what a service does from how the service is implemented, making SOAP-based services easier to consume and integrate.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

3.1 SOAP Communication Process

A typical SOAP communication process is:



If the response is delayed or incorrect, the problem can occur at any stage.

3.2 Possible SOAP Communication Problems

A. Message Formatting Errors

SOAP requires well-formed XML.

Incorrect:

```
<soap:Envelope>
  <soap:Body>
    <getData>
      <id>1001
    </getData>
  </soap:Body>
</soap:Envelope>
```

The <id> element is not closed.

Correct:

```
<soap:Envelope>
  <soap:Body>
    <getData>
      <id>1001</id>
    </getData>
  </soap:Body>
</soap:Envelope>
```

Other formatting problems include:

- Missing closing tags.
- Incorrect XML encoding.
- Invalid XML characters.
- Incorrect element hierarchy.
- Incorrect data types.
- Missing mandatory fields.

3.3 B. Incorrect Namespaces

SOAP heavily depends on XML namespaces.

For example:

```
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
```

If the client uses an incorrect namespace:

```
xmlns:soap="http://wrong.example.com/soap"
```

the server may fail to recognize the SOAP envelope.

Application namespaces must also match the service contract.

Example:

```
xmlns:hs="http://hospital.example.com/patient"
```

The namespace used by the client should correspond to the namespace defined by the service/WSDL.

3.4 C. Incorrect Endpoint

The client may be sending the request to the wrong URL.

For example:

Correct:

```
https://server.example.com/services/PatientService
```

Incorrect:

```
https://server.example.com/PatientService
```

Possible causes include:

- Server address changed.
- Wrong port number.
- Incorrect service path.
- DNS problems.
- Firewall blocking traffic.
- Service is down.
- Incorrect environment configuration.

3.5 D. Incorrect SOAPAction

Some SOAP services use a SOAP action to identify the operation.

Example:

SOAPAction: "getPatient"

If the server expects:

getPatient

but receives:

updatePatient

the request may be routed incorrectly or rejected.

3.6 E. WSDL Mismatch

The client may have been generated using an outdated WSDL.

For example:

Old WSDL:

getPatient(id)

New WSDL:

getPatient(patientId)

The client may send a message that no longer matches the server's contract.

Therefore, the WSDL version should be checked whenever the service interface changes.

3.7 F. Authentication and Authorization Problems

A SOAP request may fail because:

- Authentication token is missing.
- Token has expired.
- Username/password is incorrect.
- Digital signature is invalid.
- User does not have permission.
- Certificate has expired.

For example, a doctor may be authenticated but not authorized to access a particular patient's records.

3.8 G. Timeout and Network Problems

Delayed responses can be caused by:

Client



Network delay



Firewall / Proxy



SOAP Server



Slow Business Logic



Database

Possible causes:

- Network congestion.
- Server overload.
- Database query taking too long.
- Connection timeout.
- Incorrect timeout configuration.
- External service dependency.

3.9 Systematic Debugging Approach

A Level 4 debugging process should be systematic rather than simply modifying code randomly.

Step 1 – Check the Endpoint

Verify:

Protocol

Hostname

Port

Service path

Example:

<https://example.com:8443/services/DataService>

Check whether the endpoint is reachable.

Step 2 – Validate the SOAP Envelope

Verify:

Envelope



Header



Body

Check:

- XML syntax.
- SOAP namespace.
- Required header elements.
- Body structure.
- Closing tags.

Step 3 – Validate Against WSDL/XSD

Compare the request with the WSDL.

Check:

Operation name



Input message



Element names



Data types



Namespace

For example:

<id>1001</id>

must have the data type expected by the schema.

Step 4 – Inspect HTTP Headers

Check:

Content-Type

SOAPAction

Authorization

Content-Length

Host

For SOAP 1.1, a common content type is:

Content-Type: text/xml

Incorrect headers can prevent the server from correctly processing the request.

Step 5 – Check Authentication

Verify:

Username / Token



Token validity



Certificate



Authorization

Security failures should be recorded in server logs without exposing sensitive credentials.

Step 6 – Check Server Logs

Server logs should record:

Timestamp

Request ID

Operation

Response status

Fault code

Processing time

A correlation/request ID is particularly useful:

Request ID: REQ-2026-00125

The same ID can be followed through the client, SOAP server, application, and database logs.

Step 7 – Test with a Known-Good SOAP Request

Send a simple request using a SOAP testing client.

For example:

Test Request



Known Endpoint



Known Input



Expected Response

If the test request succeeds but the application request fails, the problem is likely in the client implementation.

Step 8 – Analyze SOAP Faults

A server should return meaningful SOAP faults.

Example:

<soap:Fault>

<faultcode>soap:Server</faultcode>

<faultstring>

Database temporarily unavailable

</faultstring>

<detail>

<errorCode>DB001</errorCode>

</detail>

</soap:Fault>

The client should distinguish between:

Client Error

Server Error

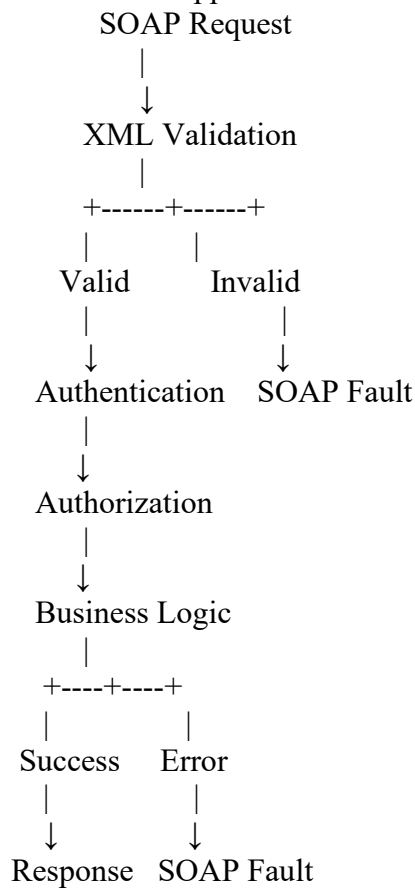
Authentication Error

Network Error

Timeout

3.10 Error Handling Strategy

A robust SOAP application should implement layered error handling.



Client-side handling

The client should:

1. Detect HTTP errors.
2. Detect SOAP faults.
3. Log the request ID.
4. Retry only appropriate transient failures.
5. Use timeout limits.
6. Display a meaningful error to the user.

For example:

Database temporarily unavailable



Retry after short interval



If still unsuccessful



Return controlled error

The system should not blindly retry every request, especially operations that create records, because this can result in duplicate transactions.

3.11 Reliability Improvements

After debugging, reliability can be improved through:

1. XML and schema validation

Ensures that requests conform to the expected structure.

2. WSDL-based development

Ensures that clients and servers follow the same contract.

3. Proper namespace management

Prevents operation and element identification problems.

4. HTTPS

Protects communication against interception.

5. Authentication and authorization

Ensures that only permitted users access services.

6. Timeout configuration

Prevents clients from waiting indefinitely.

7. Controlled retries

Useful for temporary network/server failures.

8. SOAP Fault handling

Provides structured error information.

9. Logging and monitoring

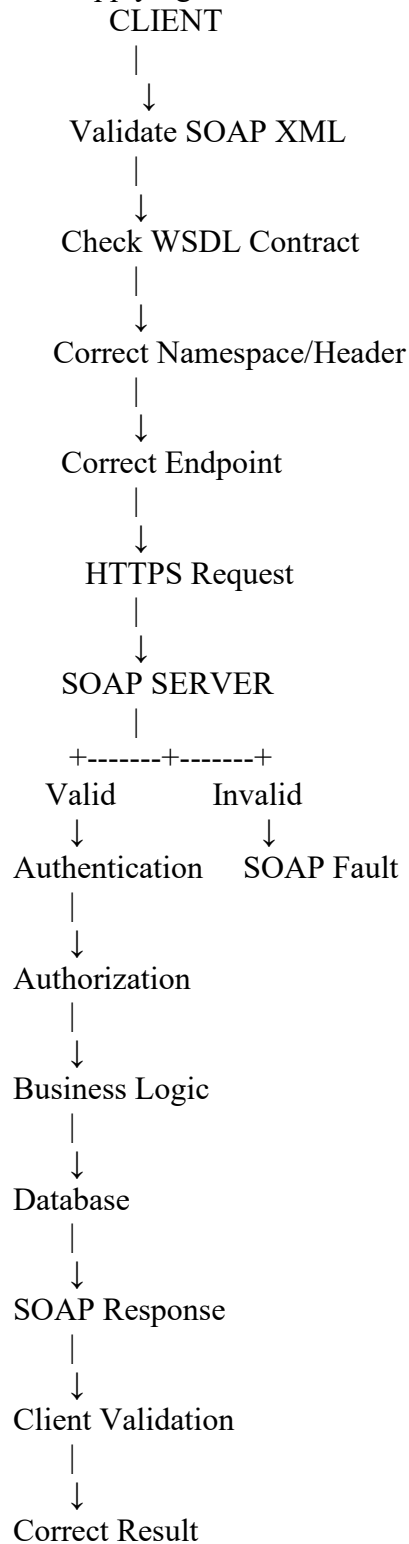
Makes failures traceable.

10. Correlation IDs

Allows a single request to be tracked across multiple systems.

3.12 Corrected Communication Flow

After applying these corrections:



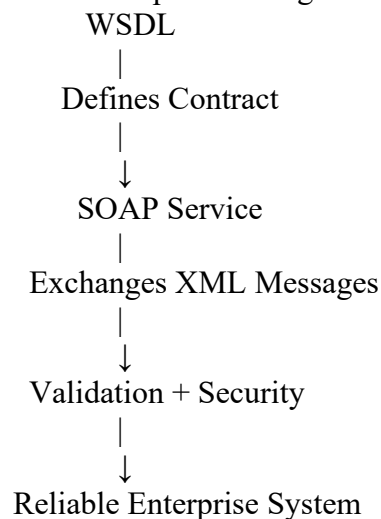
This process ensures that the system is reliable, traceable, secure, and easier to maintain.

Overall Comparison and Justification

Feature	SOAP	WSDL	Debugging/Validation
Main purpose	Message communication	Service contract	Identify and correct failures
Format	XML	XML	XML/WSDL/HTTP
Security	WS-Security, HTTPS	Can describe bindings/security-related details	Validate authentication details
Interoperability	Very high	Very high	Ensures compatibility
Error handling	SOAP Fault	Defines operation/message structure	Fault analysis
Reliability	Enterprise messaging support	Formal contract	Validation, retries, logging
Integration	Service communication	Service discovery/consumption	Ensures correct communication

Final Conclusion

The three concepts work together:



SOAP provides the structured communication mechanism, WSDL provides the formal machine-readable service contract, and validation/debugging mechanisms ensure that requests and responses are correctly processed.

For a hospital or enterprise environment, this combination is especially valuable because it provides interoperability, formal contracts, structured error handling, security mechanisms, reliability, and maintainability across multiple departments and heterogeneous systems.